PONTIFÍCIA UNIVERSIDADE CATÓLICA DE GOIÁS
Curso de Graduação

Brunno Alexcson de Carvalho

Estudo de Caso 1
Cálculo de π e Análise de Erro

Goiânia
2026

Brunno Alexcson de Carvalho

Estudo de Caso 1
Cálculo de π e Análise de Erro

Trabalho de AED apresentado no Curso de Ciências da Computação do Campus I da Pontifícia Universidade Católica de Goiás (PUC-GO).

Orientador: Prof. Andre Luiz Cardoso da Silva

Goiânia
2026

RESUMO

Este trabalho apresenta a aplicação do método de Monte Carlo para estimar o valor da constante matemática π. Foram gerados N pontos aleatórios uniformemente distribuídos no quadrado unitário [0,1] × [0,1], verificando-se quantos deles pertencem ao quarto de circunferência de raio 1. A estimativa de π foi calculada com base na proporção entre pontos internos e o total de pontos gerados. Também foram analisados o erro absoluto, erro relativo e erro percentual. Os resultados demonstram a convergência do método à medida que N aumenta, evidenciando as características estatísticas do método.

Palavras-chaves: Monte Carlo; Simulação π; Método Numérico; Convergência

ABSTRACT

This work presents the application of the Monte Carlo method to estimate the value of the mathematical constant π. N random points were generated uniformly distributed in the unit square [0,1] × [0,1], verifying how many of them belong to the quarter circle of radius 1. The estimate of π was calculated based on the ratio between internal points and the total number of points generated. The absolute error, relative error, and percentage error were also analyzed. The results demonstrate the convergence of the method as N increases, highlighting the statistical characteristics of the method.

Keywords: Monte Carlo; π Simulation; Numerical Method; Convergence

SUMÁRIO

1. INTRODUÇÃO ---------- 05
2. DESCRIÇÃO DO TEXTO ---------- 06
3. METODOLOGIA / DESENVOLVIMENTO ---------- 07
3.1 FUNDAMENTAÇÃO TEÓRICA ---------- 07
3.2 IMPLEMENTAÇÃO COMPUTACIONAL ---------- 08
4. APRESENTAÇÃO E ANÁLISE DOS RESULTADOS ---------- 09
5. DISCUSSÃO ---------- 11
6. LIMITAÇÕES E MELHORIAS ---------- 12
7. CONCLUSÃO ---------- 13
REFERÊNCIAS ---------- 14
APÊNDICE A – CÓDIGO FONTE DA SIMULAÇÃO ---------- 15
APÊNDICE B – GRÁFICOS E TABELAS COMPLEMENTARES ---------- 17

1. INTRODUÇÃO

O cálculo do valor de π é um dos problemas clássicos da matemática. Diversos métodos determinísticos e probabilísticos foram desenvolvidos ao longo da história para sua aproximação.

O método de Monte Carlo baseia-se na geração de números aleatórios para estimar resultados numéricos. Neste trabalho, utilizou-se esse método para estimar π a partir da razão entre a área de um quarto de circunferência de raio 1 e a área de um quadrado unitário.

O objetivo deste estudo é implementar computacionalmente o método, analisar sua precisão e avaliar a convergência da estimativa conforme o número de pontos gerados aumenta.

2. DESCRIÇÃO DO CONTEXTO

O experimento foi realizado utilizando a linguagem Python, com apoio das bibliotecas:

- random (geração de números aleatórios)
- math (constante π)
- matplotlib (visualização gráfica)

O ambiente utilizado foi uma IDE para Python, o PyCharm, com suporte a bibliotecas científicas.

3. METODOLOGIA / DESENVOLVIMENTO

3.1 FUNDAMENTAÇÃO TEÓRICA

Considere um quadrado de lado 1 e um quarto de circunferência de raio 1 inscrito nesse quadrado.

A área do quarto de circunferência é:

$$A = \frac{\pi}{4}$$

Se N pontos forem gerados aleatoriamente no quadrado, a razão:

$$\frac{N_{acertos}}{N}$$

aproxima a razão entre as áreas:

$$\frac{\pi}{4}$$

Logo, a estimativa de π é dada por:

$$\pi \approx 4 \cdot \frac{N_{acertos}}{N}$$

3.2 IMPLEMENTAÇÃO COMPUTACIONAL

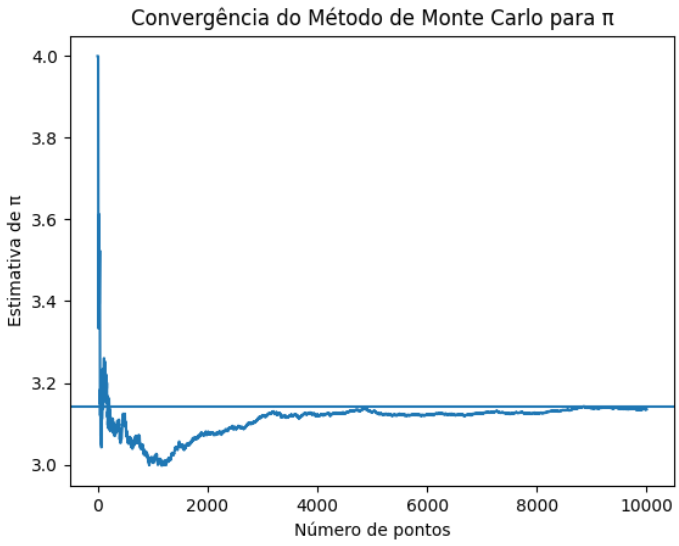
O algoritmo executa os seguintes passos:

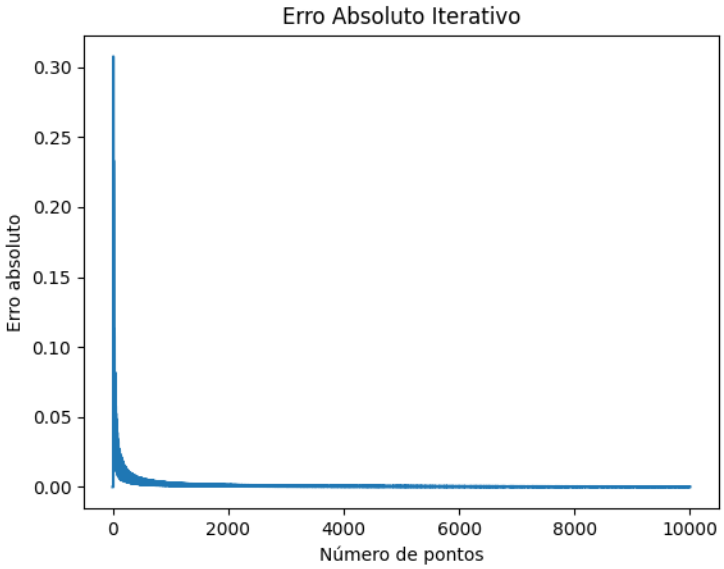
1. Gerar N pares aleatórios (x, y) ∈ [0,1]
2. Verificar se $x^2 + y^2 \leq 1$
3. Contabilizar os pontos dentro da região
4. Calcular a estimativa de π
5. Calcular erro absoluto, erro relativo e erro percentual
6. Plotar gráficos de convergência e erro

4. APRESENTAÇÃO E ANÁLISE DOS RESULTADOS

Para N = 10000 pontos, obteve-se:

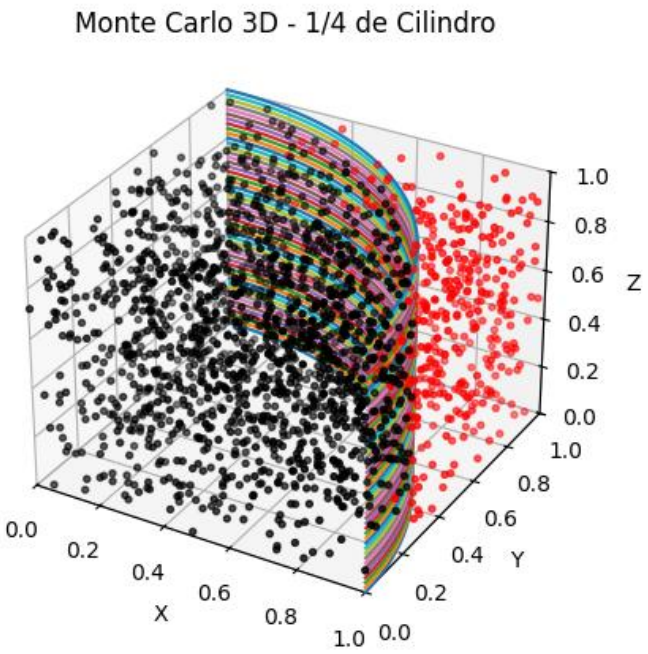
- Valor real de π: 3,141592653589793
- Valor aproximado: 3.1352
- Erro absoluto: 8.648864886495744e-05
- Erro relativo: 2.7586325869149474e-05
- Erro percentual: 0.00275863258691494 76

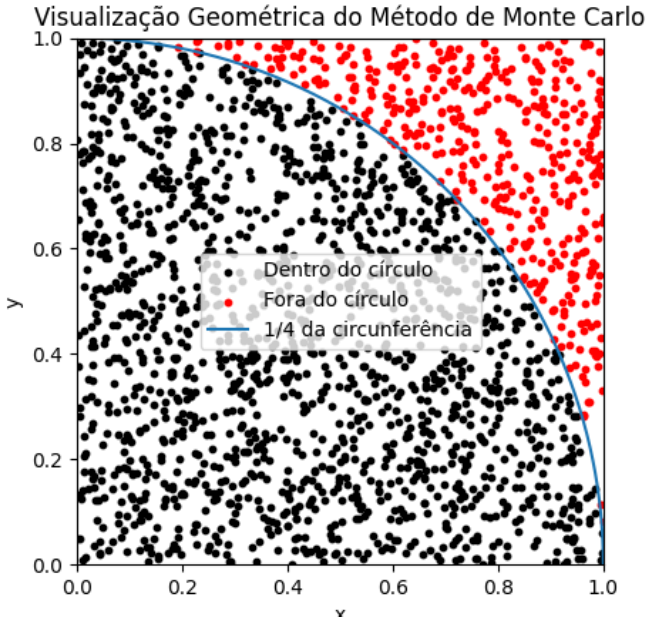




Os gráficos demonstram:

- Convergência gradual da estimativa
- Redução do erro com o aumento de N
- Distribuição visual dos pontos no quadrado





5. DISCUSSÃO

O método de Monte Carlo apresentou convergência para o valor real de π conforme o número de pontos aumentou.

Entretanto, observou-se que:

- A convergência é lenta
- Pequenas oscilações ocorrem devido à natureza probabilística
- O erro decresce aproximadamente na ordem de:

$$O\left(\frac{1}{\sqrt{N}}\right)$$

Isso ocorre porque o método depende de variáveis aleatórias e está sujeito a variações estatísticas.

6. LIMITAÇÕES E MELHORIAS

Limitações:

- Convergência lenta
- Dependência de números aleatórios
- Alto custo computacional para grande precisão

Melhorias possíveis:

- Aumentar N
- Utilizar geradores pseudoaleatórios mais sofisticados
- Aplicar técnicas de redução de variância

7. CONCLUSÃO

O estudo demonstrou que o método de Monte Carlo é eficaz para estimar π, embora apresente convergência relativamente lenta. A implementação computacional permitiu analisar a evolução da estimativa e dos erros associados, consolidando conceitos de simulação numérica e análise de erro.

Os objetivos propostos foram atingidos, sendo possível verificar experimentalmente o comportamento estatístico do método.

REFERÊNCIAS

Exemplo em formato ABNT:

- ROBERT, Christian P.; CASELLA, George. *Monte Carlo Statistical Methods*. 2. ed. New York: Springer, 2004.

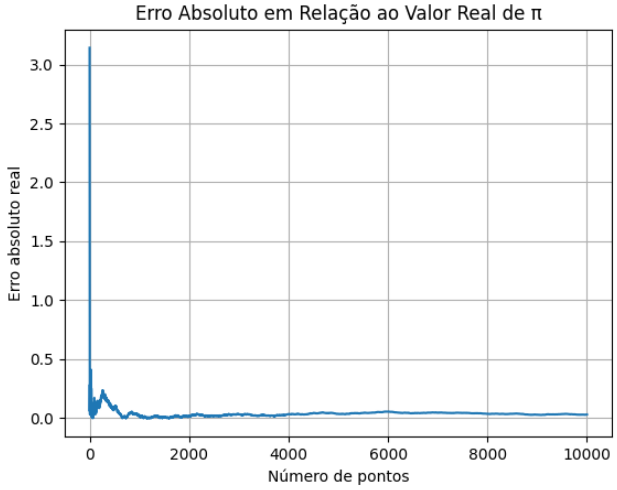
- Material didático da disciplina de Fundamentos de Computação IV.

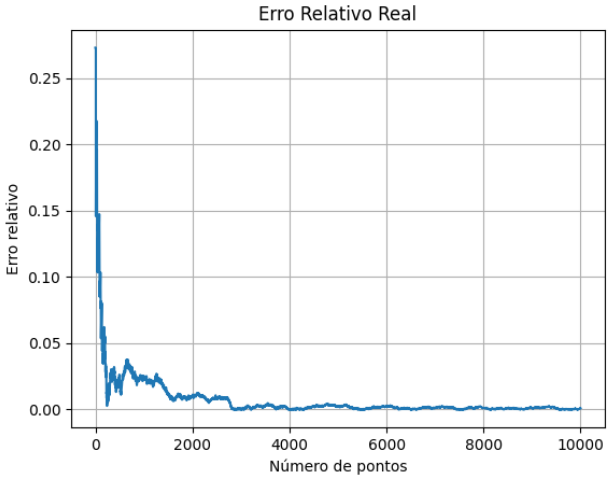
APÊNDICE A – Código Fonte da Simulação

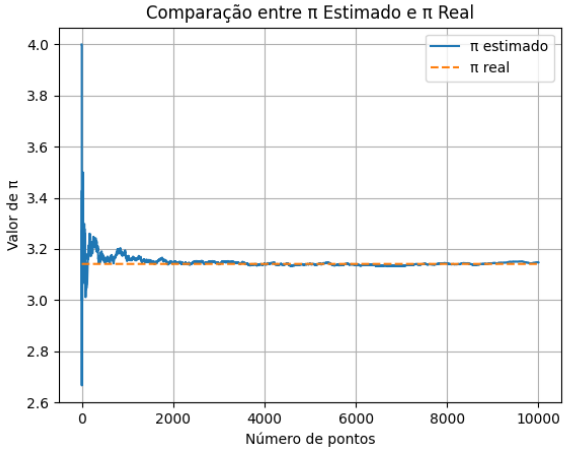
```
import random
import math
import matplotlib.pyplot as plt

def valor_pi(N):
    nacertos = 0

    # Listas para armazenar evolução (para utilizar na geração de gráficos)
    lista_i = []
    lista_pi = []
    lista_erro_abs = []
    lista_erro_rel = []
    lista_erro_rel_perc = []

    pi_anterior = 0

    for i in range(1, N+1):
        # Gera ponto aleatório no quadrado [x,y]
        x = random.random()
        y = random.random()

        # Verifica se está dentro do quarto de círculo
        if x*x + y*y <= 1:
            nacertos += 1

        # Estimativa atual de pi
        pi_atual = 4 * nacertos / i

        # Só calcula erro depois da segunda iteração
        if i > 1:
            erro_abs = abs(pi_atual - pi_anterior)
            erro_rel = erro_abs / abs(pi_atual)
            erro_rel_perc = erro_rel * 100
        else:
            erro_abs = 0
            erro_rel = 0
            erro_rel_perc = 0

        # Armazenar esses valores nas listas definidas
        lista_i.append(i)
        lista_pi.append(pi_atual)
        lista_erro_abs.append(erro_abs)
        lista_erro_rel.append(erro_rel)
        lista_erro_rel_perc.append(erro_rel_perc)

        pi_anterior = pi_atual

    return lista_i, lista_pi, lista_erro_abs, lista_erro_rel, lista_erro_rel_perc
```

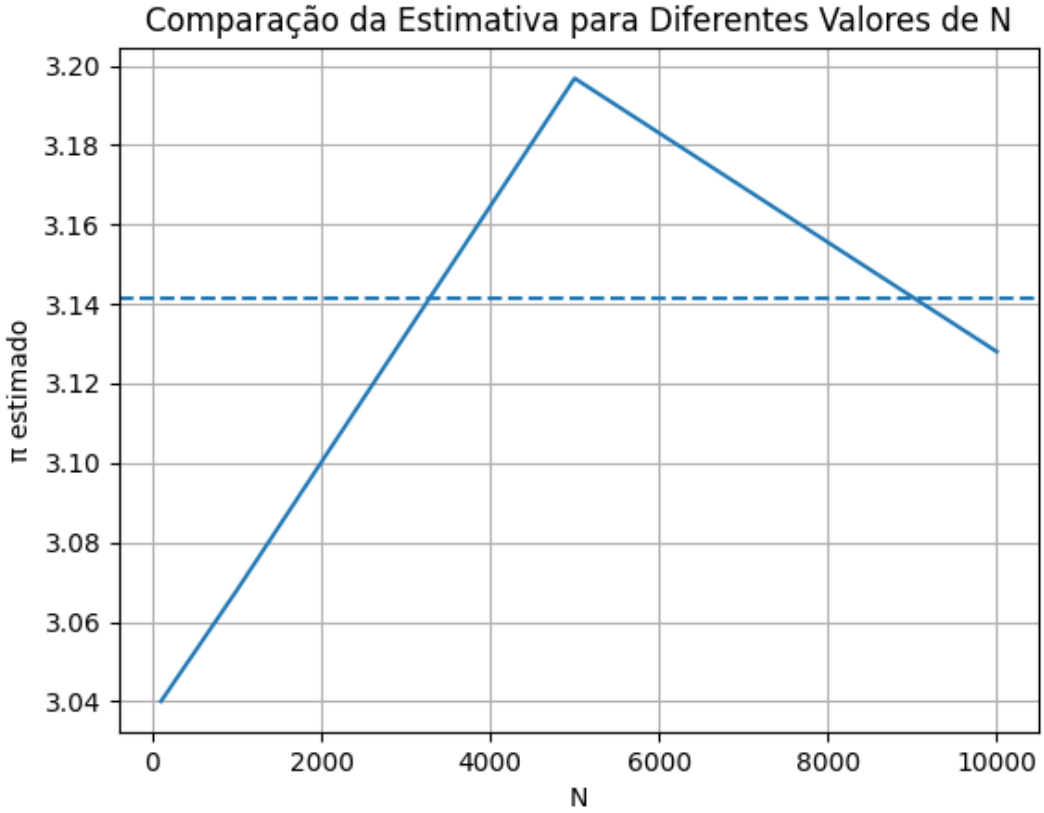
```
# Programa Principal
N = 100000 # Esse N representa o número de pontos que serão gerados para o cálculo da
aproximação de pi. Quanto maior, mais próximo será.

lista_i, lista_pi, lista_erro_abs, lista_erro_rel, lista_erro_rel_perc = valor_pi(N)

print("===== Estudo de Caso 1: Cálculo de pi e Análise de Erro =====")
print("Quantidade de pontos gerados: ", N)
print("Valor real de pi: ", math.pi)
print("Valor do pi aproximado: ", lista_pi[-1])
print("Erro Absoluto = ", lista_erro_abs[-1])
print("Erro Relativo = ", lista_erro_rel[-1])
print("Erro Relativo Percentual = ", lista_erro_rel_perc[-1])


# Gráfico para analisar o valor de pi_aprox em função do número de pontos
plt.figure()
plt.plot(lista_i, lista_pi)
plt.axhline(math.pi)
plt.title("Convergência do Método de Monte Carlo para π")
plt.xlabel("Número de pontos")
plt.ylabel("Estimativa de π")
plt.show()

# Gráfico do erro absoluto
plt.figure()
plt.plot(lista_i, lista_erro_abs)
plt.title("Erro Absoluto Iterativo")
plt.xlabel("Número de pontos")
plt.ylabel("Erro absoluto")
plt.show()
```

APÊNDICE B – Gráficos e Tabelas Complementares








Comparação da Estimativa para Diferentes Valores de N
π estimado
N
3.20
3.18
3.16
3.14
3.12
3.10
3.08
3.06
3.04
0
2000
4000
6000
8000
10000